

Machine Learning for Physicists

Day 2: Generative models

Sascha Cassandra Diefenbacher, Heidelberg University

Sources. These are lecture notes for the second day of a four-day lecture series. Large parts are taken from, or closely follow, the lecture notes *Modern Machine Learning for LHC Physicists* by T. Plehn, A. Butter, B. Dillon, T. Heimel, C. Krause, and R. Winterhalder, Ref. [1], whose notation we mostly follow.

- Passages taken from Ref. [1] are marked with a grey bar and the section they are taken from. Omissions are marked by [...]; notation and cross-references are adapted to these notes.
- Passages which follow Ref. [1] but are rewritten are marked as *adapted*, again with the section.
- Where we deviate from Ref. [1] to correct an error, this is stated in a footnote.
- Figures from other publications are cited in their captions.

We quote and cite the updated version of Ref. [1] from June 2026, available at https://www.thphys.uni-heidelberg.de/~plehn/pics/modern_ml.pdf; its section numbers and wording differ in parts from the arXiv versions. Ref. [1] is distributed under the Creative Commons Attribution 4.0 International license (CC BY 4.0). The presentation of diffusion models follows the author's lecture on generative models at the ML4FP school 2025, and the original papers on coupling layers [2], denoising diffusion [3], and flow matching [4].

Use of generative AI. These notes were prepared with the help of a generative AI assistant (Claude, Anthropic). It was used to compile the passages from the sources above, to draft parts of the remaining text and the step-by-step derivations, and to write the code for the figures without a cited source. All content was reviewed and edited by the author, who is responsible for any remaining errors.

1 Generative models as density estimators

Yesterday the network learned a function with a target for each training point. Generative networks learn a density instead.

The concept which allows us to apply modern machine learning to LHC event generation and simulations is generative networks. To train them, we start with a dataset which implicitly encodes a probability density over a physics or phase space. A generative network learns this underlying density and maps it to a latent space from which we can then sample using for example flat or Gaussian random numbers,

$$r \sim p_{\text{latent}}(r) \quad \rightarrow \quad x = G_{\theta}(r) \sim p_{\text{model}}(x) \approx p_{\text{data}}(x) . \quad (1.1)$$

The last step represents the network training, for instance in terms of a variational approximation. A typical latent distribution is the standard multi-dimensional Gaussian, $p_{\text{latent}}(r) = \mathcal{N}(0, 1)$. [...] The generated data should then have the same form as the training data, in which case the generative network will produce statistically independent samples reproducing the implicit underlying structures of the training data. Because we train on a distribution of events and there are no labels or any other truth information about the learned phase space density, generative network training is considered unsupervised.

— taken from Ref. [1], Sec. 4, *Generation and simulation*

We need generative networks whenever a density is expensive to sample, to evaluate, or both:

- *event generation*: learn and sample LHC phase space densities, $\mathcal{O}(10)$ dimensions with variable multiplicity [5];
- *detector simulation*: fast calorimeter showers replacing Geant4, now used in ATLAS [6] and benchmarked in the CaloChallenge [7];
- *anomaly detection*: learn the background density in a side band and interpolate into the signal region [8];
- *unfolding and inference*: conditional densities $p(x_{\text{part}}|x_{\text{reco}})$ or $p(\text{params}|\text{data})$;
- *beyond particle physics*: sampling lattice field configurations [9], gravitational-wave parameter inference [10], cosmological emulators, molecular conformations.

We want two things from such a model: generate new samples, $r \rightarrow x$, and evaluate the density of a given sample, $x \rightarrow p_{\text{model}}(x)$. The first gives us events, the second lets us train the model with a likelihood.

2 Normalizing flows

2.1 From noise to a density

In Eq.(1.1) we generate events by transforming latent noise, $x = G_\theta(r)$. To train such a generator we use the likelihood recipe of Day 1: with a flat prior and independent training events x_j ,

$$\hat{\theta} \simeq \operatorname{argmax}_\theta \prod_j p_{\text{model}}(x_j|\theta) \quad \Rightarrow \quad \boxed{\mathcal{L} = -\frac{1}{N} \sum_j \log p_{\text{model}}(x_j|\theta) \longrightarrow -\left\langle \log p_{\text{model}}(x|\theta) \right\rangle_{p_{\text{data}}}}, \quad (2.1)$$

where the average over the training events becomes an expectation value for a large training set. There is no target this time, the likelihood is the model density evaluated at the training events. So the question of this section is: which density $p_{\text{model}}(x|\theta)$ does the transformation of noise produce, and can we evaluate it for a given event x ?

Why not simply let a network output the density? A positive network output $f_\theta(x) \geq 0$ is flexible enough, but it is not normalized, and without the normalization the loss has no minimum. Scaling the output by a constant c changes the loss by

$$-\left\langle \log(c f_\theta(x)) \right\rangle_{p_{\text{data}}} = -\log c - \left\langle \log f_\theta(x) \right\rangle_{p_{\text{data}}} \xrightarrow{c \rightarrow \infty} -\infty, \quad (2.2)$$

so the training would just make the function large everywhere. We could divide by $Z(\theta) = \int dx f_\theta(x)$, but this integral over a high-dimensional phase space would have to be recomputed after every update; already a grid with 10 points per direction needs 10^D network evaluations. And sampling from $f_\theta(x)/Z(\theta)$ would still be hard.

Starting from noise avoids both problems. A standard Gaussian is normalized and trivial to sample, and transforming its samples gives samples of the transformed density, which stays normalized because no probability is lost on the way. What we need is a way to compute this transformed density for a given event, and that requires two things. We have to find the latent point r which produced x , so the transformation has to be invertible. And we have to know how the transformation stretches and compresses volumes, which is its Jacobian.

For the first requirement we choose the dimensionality of the latent vector r the same as the dimensionality of the phase space vector x . This gives us the opportunity to define the generator as a bijection mapping, which means that the network evaluated in the opposite direction maps data to noise. This architecture is called a normalizing flow or an invertible neural network (INN),

$$\text{latent } r \sim p_{\text{latent}} \quad \begin{array}{c} \xrightarrow{G_\theta(r)} \\ \xleftarrow{\bar{G}_\theta(x)} \end{array} \quad \text{phase space } x \sim p_{\text{data}}, \quad (2.3)$$

where $\bar{G}_\theta(x)$ denotes the inverse transformation to $G_\theta(r)$. Given a sample r from the latent distribution, we can use G to generate a sample from the target distribution. Alternatively, we can use a sample x from the target distribution to compute its density using the inverse direction. [adapted from Ref. [1], Sec. 4.3.1, *Architecture*]

To relate the two densities we start in one dimension with a monotonic map $x = G(r)$. Probability is conserved: the events in a small interval dr around r end up in the interval dx around $x = G(r)$,

$$p_{\text{model}}(x) |dx| = p_{\text{latent}}(r) |dr| \quad \Rightarrow \quad p_{\text{model}}(x) = p_{\text{latent}}(r) \left| \frac{dr}{dx} \right| = p_{\text{latent}}(r) \left| \frac{dG(r)}{dr} \right|^{-1} = p_{\text{latent}}(\bar{G}(x)) \left| \frac{d\bar{G}(x)}{dx} \right|, \quad (2.4)$$

where the last two forms use the derivative of the inverse function, $d\bar{G}/dx = (dG/dr)^{-1}$. A simple example: for $r \sim \mathcal{U}[0, 1]$ and $x = G(r) = 2r + 1$ we find $p_{\text{model}}(x) = 1 \cdot |dG/dr|^{-1} = 1/2$ on $x \in [1, 3]$. Stretching the interval by a factor two halves the density, so that it stays normalized.

In D dimensions we need the generalization of the slope dG/dr . It is the Jacobian matrix of the map, the matrix of all first derivatives,

$$J_\theta(r) \equiv \frac{\partial G_\theta(r)}{\partial r} \quad \text{with entries} \quad (J_\theta)_{ij} = \frac{\partial G_{\theta,i}(r)}{\partial r_j}, \quad i, j = 1, \dots, D, \quad (2.5)$$

which for $D = 1$ is just dG/dr . We need two of its properties. First, it describes the map locally: a small displacement δ around r is mapped to

$$G_\theta(r + \delta) \approx G_\theta(r) + J_\theta(r) \delta. \quad (2.6)$$

A small box around r with edges $dr_1 e_1, \dots, dr_D e_D$ along the unit vectors is therefore mapped to a slanted box, spanned by the columns of J_θ , each scaled by its dr_j . The volume spanned by D vectors is the absolute value of the determinant of the matrix with these vectors as columns, so the volumes before and after the map are related by

$$dx = |\det J_\theta(r)| dr \quad \text{with} \quad dr = dr_1 \cdots dr_D. \quad (2.7)$$

The determinant takes the role of $|dG/dr|$ in one dimension: it is the factor by which the map stretches or compresses volumes. Second, the Jacobian of the inverse map is the inverse matrix. Differentiating $\bar{G}_\theta(G_\theta(r)) = r$ with the chain rule gives

$$\frac{\partial \bar{G}_\theta(x)}{\partial x} J_\theta(r) = \mathbb{1} \quad \Rightarrow \quad \frac{\partial \bar{G}_\theta(x)}{\partial x} = J_\theta(r)^{-1} \quad \Rightarrow \quad \det \frac{\partial \bar{G}_\theta(x)}{\partial x} = \frac{1}{\det J_\theta(r)}, \quad (2.8)$$

at $x = G_\theta(r)$, the D -dimensional version of $d\bar{G}/dx = (dG/dr)^{-1}$. Now we plug the Jacobian into probability conservation: the events in the small box around r end up in the slanted box around x ,

$$\begin{aligned} p_{\text{model}}(x) dx &= p_{\text{latent}}(r) dr \\ \Leftrightarrow p_{\text{model}}(x) |\det J_\theta(r)| dr &= p_{\text{latent}}(r) dr \\ \Leftrightarrow p_{\text{model}}(x) &= p_{\text{latent}}(r) |\det J_\theta(r)|^{-1} = p_{\text{latent}}(\bar{G}_\theta(x)) \left| \det \frac{\partial \bar{G}_\theta(x)}{\partial x} \right|. \end{aligned} \quad (2.9)$$

This is the change of variables formula from any statistics course: the flow is a learned, very flexible coordinate transformation, and the Jacobian keeps track of how it stretches and compresses volumes.

[adapted from Ref. [1], Sec. 4.3.1, *Architecture*, with the derivation added]

This leaves the second requirement. The latent Gaussian takes care of normalization and sampling, but we have to evaluate the Jacobian determinant for every training event in every update, and for a general $D \times D$ matrix this costs of the order of D^3 operations. So we need transformations which are flexible, but whose Jacobian determinant is cheap.

The construction of G_θ relies on the usual assumption that a chain of simple invertible nonlinear maps gives us a complex map. This means we transform the latent space into phase space with several transformation layers, for which we need to know the Jacobians. For instance, we can use affine coupling layers as building blocks. Here, the input vector r is split in half, $r = (r_1, r_2)$, allowing us to compute the output $x = (x_1, x_2)$ of the layer as

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} r_1 \odot e^{s_2(r_2)} + t_2(r_2) \\ r_2 \odot e^{s_1(x_1)} + t_1(x_1) \end{pmatrix} \quad \Leftrightarrow \quad \begin{pmatrix} r_1 \\ r_2 \end{pmatrix} = \begin{pmatrix} (x_1 - t_2(r_2)) \odot e^{-s_2(r_2)} \\ (x_2 - t_1(x_1)) \odot e^{-s_1(x_1)} \end{pmatrix}, \quad (2.10)$$

where s_i, t_i ($i = 1, 2$) are arbitrary functions, and $\odot$ is the element-wise product. In practice each of them will be a small multi-layer network.

— taken from Ref. [1], Sec. 4.3.1, *Architecture*

This construction goes back to the RealNVP architecture [2]. The networks s and t never have to be inverted. The inverse is computed in the opposite order: from $x = (x_1, x_2)$ we first get $r_2 = (x_2 - t_1(x_1)) \odot e^{-s_1(x_1)}$, which only needs x_1 , and then $r_1 = (x_1 - t_2(r_2)) \odot e^{-s_2(r_2)}$ with the r_2 we just computed.

To compute the Jacobian we split the layer into its two steps.¹ The first step changes only r_1 , the second only r_2 ,

$$\begin{pmatrix} r_1 \\ r_2 \end{pmatrix} \xrightarrow{\text{step 1}} \begin{pmatrix} x_1 = r_1 \odot e^{s_2(r_2)} + t_2(r_2) \\ r_2 \end{pmatrix} \xrightarrow{\text{step 2}} \begin{pmatrix} x_1 \\ x_2 = r_2 \odot e^{s_1(x_1)} + t_1(x_1) \end{pmatrix}. \quad (2.11)$$

Each step has a triangular Jacobian, because the part that is passed through unchanged contributes a unit block and a zero block,

$$J_1 = \begin{pmatrix} \text{diag}(e^{s_2(r_2)}) & \partial x_1 / \partial r_2 \\ 0 & \mathbb{1} \end{pmatrix} \quad \text{and} \quad J_2 = \begin{pmatrix} \mathbb{1} & 0 \\ \partial x_2 / \partial x_1 & \text{diag}(e^{s_1(x_1)}) \end{pmatrix}. \quad (2.12)$$

The off-diagonal blocks contain the derivatives of the networks s and t , but the determinant of a triangular matrix is the product of its diagonal entries, so we never need them. The full layer is the chain of the two steps, $J_\theta(r) = J_2 J_1$, and the determinant of a product is the product of the determinants,

$$\det J_\theta(r) = \det J_2 \det J_1 = \prod_i e^{s_{1,i}(x_1)} \prod_i e^{s_{2,i}(r_2)} \Rightarrow \log |\det J_\theta(r)| = \sum_i s_{1,i}(x_1) + \sum_i s_{2,i}(r_2). \quad (2.13)$$

Such a Jacobian determinant is computationally inexpensive and still allows for complex transformations. We refer to the sequence of coupling layers as $G_\theta(r)$, collecting the parameters of the individual nets s, t into a joint θ .

— taken from Ref. [1], Sec. 4.3.1, *Architecture*

For a chain of layers the Jacobian is again a product, and the log-determinants of all layers add up.

2.2 Training the flow

Now we can evaluate the likelihood loss of Eq.(2.1) for a flow. Inserting the change of variables, Eq.(2.9), gives

$$\mathcal{L}_{\text{INN}} = - \left\langle \log p_{\text{model}}(x|\theta) \right\rangle_{p_{\text{data}}} = - \left\langle \log p_{\text{latent}}(\bar{G}_\theta(x)) + \log \left| \det \frac{\partial \bar{G}_\theta(x)}{\partial x} \right| \right\rangle_{p_{\text{data}}}. \quad (2.14)$$

With Eq.(2.9) in its first form, or equivalently with $\det \partial \bar{G}_\theta / \partial x = 1 / \det J_\theta(r)$ from the inverse of the Jacobian, we can write the same loss in terms of the Jacobian of the generator, evaluated at the latent point $r = \bar{G}_\theta(x)$ of each training event,

$$\mathcal{L}_{\text{INN}} = - \left\langle \log p_{\text{latent}}(\bar{G}_\theta(x)) - \log \left| \det J_\theta(\bar{G}_\theta(x)) \right| \right\rangle_{p_{\text{data}}}. \quad (2.15)$$

This is the form in which the coupling layers deliver the Jacobian, Eq.(2.13). [adapted from Ref. [1], Sec. 4.3.1, *Architecture*]

As on Day 1, minimizing this loss is the same as minimizing the KL-divergence between the true but unknown phase space distribution and the model, because the two only differ by the entropy of the data, which does not depend on θ ,

$$D_{\text{KL}}[p_{\text{data}}, p_{\text{model}}] = \left\langle \log \frac{p_{\text{data}}(x)}{p_{\text{model}}(x|\theta)} \right\rangle_{p_{\text{data}}} = \underbrace{\left\langle \log p_{\text{data}}(x) \right\rangle_{p_{\text{data}}}}_{\text{const}(\theta)} - \left\langle \log p_{\text{model}}(x|\theta) \right\rangle_{p_{\text{data}}}. \quad (2.16)$$

Next to the losses of Day 1 the three cases differ only in what we model the probability of,

$$\begin{aligned} \text{regression:} \quad & p(y|x, \theta) = \mathcal{N}(y; f_\theta(x), \sigma) \Rightarrow \text{MSE}, \\ \text{classification:} \quad & P_\theta(y|x) = C_\theta(x)^y (1 - C_\theta(x))^{1-y} \Rightarrow \text{BCE}, \\ \text{density:} \quad & p_{\text{model}}(x) = p_{\text{latent}}(\bar{G}_\theta(x)) \left| \det \partial \bar{G}_\theta / \partial x \right| \Rightarrow \text{negative log-likelihood}. \end{aligned} \quad (2.17)$$

¹Ref. [1], Sec. 4.3.1, writes the Jacobian of the full layer as a single upper triangular matrix with a vanishing lower-left block. Since x_2 depends on r_1 through x_1 , that block does not vanish; the determinant given there is still correct, because the layer is a product of two triangular steps.

For a standard Gaussian latent space in D dimensions, and with $r = \bar{G}_\theta(x)$,

$$-\log p_{\text{latent}}(r) = -\log \left[\frac{1}{(2\pi)^{D/2}} e^{-|r|^2/2} \right] = \frac{|r|^2}{2} + \frac{D}{2} \log(2\pi) . \quad (2.18)$$

Inserting this and the log-determinant of Eq.(2.13) into Eq.(2.15), for one coupling layer the loss becomes

$$\mathcal{L}_{\text{INN}} = \left\langle \frac{|\bar{G}_\theta(x)|^2}{2} + \sum_i s_{1,i}(x_1) + \sum_i s_{2,i}(r_2) \right\rangle_{p_{\text{data}}} + \text{const}(\theta) , \quad (2.19)$$

with the sums running over all layers for a chain of coupling layers. The first term pulls every data point towards the origin of latent space. On its own it would be minimized by compressing all data into a single point, but compressing the data in the direction $x \rightarrow r$ means expanding in the direction $r \rightarrow x$, large values of s , which the second term penalizes. The balance between the two is the correct density. We train in the direction data $\rightarrow$ noise and generate in the direction noise $\rightarrow$ data.

3 Diffusion models

3.1 Destroying structure is easy

The flow learns the map between noise and data in one go, and it pays for this with a constrained, invertible architecture. Diffusion models split the problem into many small steps. The key observation is that one direction is trivial: we can destroy all structure in the data by adding a little Gaussian noise many times, as in Fig. 1. If a network can learn to undo *one small step* of this noising, we can chain these steps and walk all the way back from noise to data,

$$\text{data } x_0 \xrightleftharpoons[\leftarrow \text{learned denoising}]{\rightarrow \text{add noise}} \text{noise } x_T \sim \mathcal{N}(0, 1) . \quad (3.1)$$

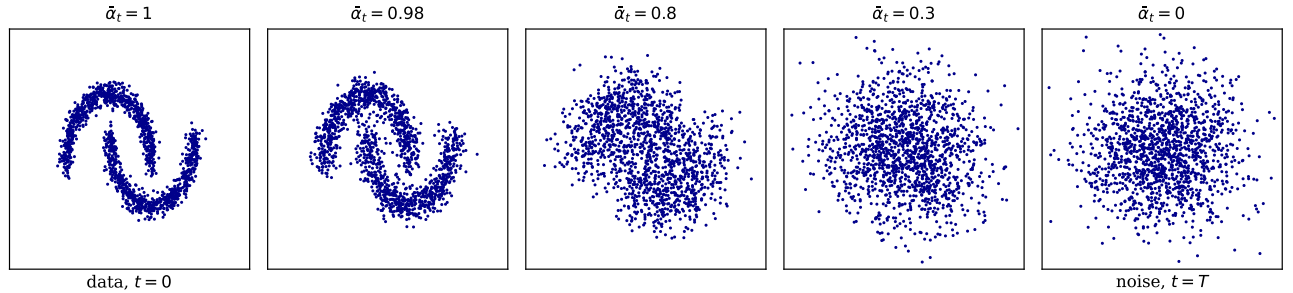


Figure 1: Forward diffusion of the two-moons data: the same events after more and more noise has been added, from data (left) to a standard Gaussian (right).

The forward direction adds noise with a small variance β_t in each step $t = 1, \dots, T$,²

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon \quad \text{with } \epsilon \sim \mathcal{N}(0, 1) . \quad (3.2)$$

Naively, we would just add the noise, but then every step would broaden the distribution. For standardized data, with mean zero and unit variance, the shrinking by $\sqrt{1 - \beta_t}$ compensates this exactly. Independent random numbers add their variances, so

$$\text{Var}(x_t) = (1 - \beta_t) \text{Var}(x_{t-1}) + \beta_t \text{Var}(\epsilon) = (1 - \beta_t) \cdot 1 + \beta_t \cdot 1 = 1 , \quad (3.3)$$

²We use $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t$ as in Ref. [3]; Ref. [1], Sec. 4.4.1, writes $\bar{\beta}_t = 1 - \bar{\alpha}_t$.

while the information about the starting point is suppressed by a factor $\sqrt{1 - \beta_t}$ in every step. The distribution ends up as a standard Gaussian instead of growing without bound. The sequence $\beta_1, \dots, \beta_T$ is called the noise schedule. We do not have to go through all steps one by one. With $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ we show by induction that

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \quad \text{with } \epsilon \sim \mathcal{N}(0, 1). \quad (3.4)$$

For $t = 1$ this is the single step, since $\bar{\alpha}_1 = \alpha_1 = 1 - \beta_1$. If it holds for t , the next step gives

$$\begin{aligned} x_{t+1} &= \sqrt{\bar{\alpha}_{t+1}} x_t + \sqrt{\beta_{t+1}} \epsilon' = \sqrt{\bar{\alpha}_{t+1}} (\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon) + \sqrt{\beta_{t+1}} \epsilon' \\ &= \sqrt{\bar{\alpha}_{t+1}} x_0 + \underbrace{\sqrt{\bar{\alpha}_{t+1}(1 - \bar{\alpha}_t)} \epsilon + \sqrt{1 - \bar{\alpha}_{t+1}} \epsilon'}_{\text{Gaussian, variance } \alpha_{t+1} - \bar{\alpha}_{t+1} + 1 - \alpha_{t+1} = 1 - \bar{\alpha}_{t+1}}, \end{aligned} \quad (3.5)$$

using $\alpha_{t+1} \bar{\alpha}_t = \bar{\alpha}_{t+1}$. The sum of two independent Gaussian noise terms is a single Gaussian noise term with the summed variance, so $x_{t+1} = \sqrt{\bar{\alpha}_{t+1}} x_0 + \sqrt{1 - \bar{\alpha}_{t+1}} \epsilon''$, which is Eq.(3.4) for $t + 1$. This is all we need to know about the forward process: a noisy version of a data point at any time t is a mix of the data point and pure noise, and for $\bar{\alpha}_T \rightarrow 0$ nothing of the data point is left. [adapted from Ref. [1], Sec. 4.4.1, *Denosing diffusion probabilistic model*, with the derivation added]

3.2 Learning to denoise

Undoing a step is harder, because a noisy point x_t could have come from many different data points. Instead of learning the reverse step directly, we train a network to look at a noisy point and the time step, and to predict the noise that was added,

$$\epsilon_\theta(x_t, t) \approx \epsilon. \quad (3.6)$$

Training is a regression, exactly like on Day 1:

1. draw a data point x_0 , a time step t , and noise $\epsilon \sim \mathcal{N}(0, 1)$;
2. compute the noisy point x_t with Eq.(3.4);
3. minimize the MSE between the true and the predicted noise,

$$\mathcal{L}_{\text{DDPM}} = \left\langle |\epsilon - \epsilon_\theta(x_t, t)|^2 \right\rangle_{x_0 \sim p_{\text{data}}, t, \epsilon}. \quad (3.7)$$

There is no Jacobian and no invertible architecture: ϵ_θ can be any network, which only needs t as an additional input. This setup is called a Denosing Diffusion Probabilistic Model (DDPM) [3].

What does the network learn? The same noisy point x_t appears in the training for many different pairs (x_0, ϵ) , so no network can predict the noise exactly. The MSE then has a simple answer. For any target ϵ and any function $g(x_t)$ we add and subtract the conditional mean $\langle \epsilon | x_t \rangle$, the average noise over all training pairs which lead to the point x_t ,

$$\begin{aligned} \left\langle |\epsilon - g(x_t)|^2 \right\rangle &= \left\langle |(\epsilon - \langle \epsilon | x_t \rangle) + (\langle \epsilon | x_t \rangle - g(x_t))|^2 \right\rangle \\ &= \left\langle |\epsilon - \langle \epsilon | x_t \rangle|^2 \right\rangle + \left\langle |\langle \epsilon | x_t \rangle - g(x_t)|^2 \right\rangle + 2 \left\langle (\epsilon - \langle \epsilon | x_t \rangle) \cdot (\langle \epsilon | x_t \rangle - g(x_t)) \right\rangle. \end{aligned} \quad (3.8)$$

In the mixed term we first average over ϵ at fixed x_t , where the second bracket is a constant and the first averages to $\langle \epsilon | x_t \rangle - \langle \epsilon | x_t \rangle = 0$. The first term does not depend on g . So the MSE is minimal for

$$\epsilon_\theta(x_t, t) = \langle \epsilon | x_t \rangle, \quad (3.9)$$

the *average* noise over all data points that could have produced x_t . The first term is what remains: the loss does not go to zero even for a perfect network. Solving Eq.(3.4) for x_0 and averaging at fixed x_t turns this into the average data point compatible with x_t , our best guess,

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}} \quad \Rightarrow \quad \hat{x}_0(x_t, t) \equiv \langle x_0 | x_t \rangle = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \quad (3.10)$$

For large t almost every data point is compatible with x_t , and the best guess is close to the mean of the whole dataset, a blurry average. For small t only the nearby data points are compatible, and the guess becomes sharp. This is why we generate in small steps: we move a little towards the current best guess, add a bit of fresh noise, and guess again with a sharper picture.

3.3 Generating

We start from pure noise, $x_T \sim \mathcal{N}(0, 1)$, and repeat for $t = T, \dots, 1$

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sqrt{\beta_t} z \quad \text{with } z \sim \mathcal{N}(0, 1), \quad z = 0 \text{ in the last step.} \quad (3.11)$$

Each step removes the part of the predicted noise that belongs to this step, undoes the shrinking, and adds a little fresh noise, which keeps the generated points from all collapsing onto the same best guess. The price for the flexible network is speed: generating one event needs T network evaluations, typically hundreds.

Where the sampling step comes from. If we knew the data point x_0 , the step back from x_t would follow from Bayes' theorem for the forward process. Because each forward step only depends on the previous point, $p(x_t|x_{t-1}, x_0) = p(x_t|x_{t-1})$, and

$$p(x_{t-1}|x_t, x_0) = \frac{p(x_t|x_{t-1}) p(x_{t-1}|x_0)}{p(x_t|x_0)} \propto \exp \left(-\frac{(x_t - \sqrt{\alpha_t} x_{t-1})^2}{2\beta_t} - \frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0)^2}{2(1 - \bar{\alpha}_{t-1})} \right), \quad (3.12)$$

where the denominator does not depend on x_{t-1} . The exponent is quadratic in x_{t-1} , so this is a Gaussian in x_{t-1} . Its inverse variance is the coefficient of $-x_{t-1}^2/2$,

$$\frac{1}{\tilde{\beta}_t} = \frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1 - \bar{\alpha}_{t-1})} = \frac{1 - \bar{\alpha}_t}{\beta_t(1 - \bar{\alpha}_{t-1})} \quad \Rightarrow \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t, \quad (3.13)$$

using $\alpha_t + \beta_t = 1$ and $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$. Its mean is $\tilde{\beta}_t$ times the coefficient of the linear term,

$$\tilde{\mu}_t = \tilde{\beta}_t \left(\frac{\sqrt{\alpha_t}}{\beta_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} x_0 \right) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0. \quad (3.14)$$

We do not know x_0 , but we can express it through the noise with Eq.(3.10), $x_0 = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon) / \sqrt{\bar{\alpha}_t}$. With $\sqrt{\bar{\alpha}_{t-1}} / \sqrt{\bar{\alpha}_t} = 1 / \sqrt{\alpha_t}$ the second term becomes $\beta_t(x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon) / ((1 - \bar{\alpha}_t) \sqrt{\alpha_t})$, and collecting the coefficients of x_t ,

$$\frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} + \frac{\beta_t}{(1 - \bar{\alpha}_t) \sqrt{\alpha_t}} = \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}{(1 - \bar{\alpha}_t) \sqrt{\alpha_t}} = \frac{1}{\sqrt{\alpha_t}}, \quad (3.15)$$

we find

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right). \quad (3.16)$$

Replacing the unknown ϵ by the network prediction $\epsilon_\theta(x_t, t)$ and drawing x_{t-1} from a Gaussian around this mean gives Eq.(3.11). For the width of the step the original DDPM [3] uses β_t ; the posterior width $\tilde{\beta}_t$ works just as well, since the two only differ noticeably in the first few steps.

3.4 Conditional generation

So far we generate events from the full training distribution. Often we want events for given conditions c instead:

- calorimeter showers for a given energy and angle of the incoming particle;
- LHC events for a given number of jets;
- *unfolding*: parton-level events x_{part} for a measured reco-level event x_{reco} ;
- *inference*: model parameters for a measured dataset.

A conditional generative network learns the conditional density, generating for a given c

$$r \sim p_{\text{latent}}(r) \rightarrow x = G_{\theta}(r; c) \sim p_{\text{model}}(x|c, \theta) \approx p_{\text{data}}(x|c) . \quad (3.17)$$

The training follows the recipe of Day 1 once more. The training data are pairs (x_j, c_j) , and their joint probability factorizes into the part we model and the distribution of the conditions,

$$p(x_{\text{train}}|\theta) = \prod_j p(x_j, c_j|\theta) = \prod_j p_{\text{model}}(x_j|c_j, \theta) p(c_j) , \quad (3.18)$$

where $p(c_j)$ does not depend on θ , exactly like the inputs $p(x_j)$ on Day 1. The negative log-likelihood per event is

$$\mathcal{L}_{\text{cond}} = -\frac{1}{N} \sum_j \log p_{\text{model}}(x_j|c_j, \theta) \rightarrow -\left\langle \log p_{\text{model}}(x|c, \theta) \right\rangle_{p_{\text{data}}(x, c)} . \quad (3.19)$$

This is the structure of the regression of Day 1, with the condition c as input and the event x as output. The regression assumed a fixed Gaussian shape around the prediction, $p(y|x, \theta) = \mathcal{N}(y; f_{\theta}(x), \sigma)$, while a conditional generative network learns an arbitrary shape of the output distribution for each input.

[adapted from Ref. [1], Sec. 4.3.2, *Event generation with uncertainties*, for the loss]

The condition enters the networks as an additional input:

- *Conditional flow (cINN)*: for every fixed c the map $G_{\theta}(r; c)$ is invertible in r . The condition is passed to the subnetworks of the coupling layers, $s(r_2, c)$ and $t(r_2, c)$, but it is not transformed itself. The change of variables at fixed c is

$$p_{\text{model}}(x|c, \theta) = p_{\text{latent}}(\bar{G}_{\theta}(x; c)) \left| \det \frac{\partial \bar{G}_{\theta}(x; c)}{\partial x} \right| , \quad (3.20)$$

with derivatives with respect to x only, so the Jacobian of a coupling layer is still the product of the e^s factors of Eq.(2.13), now with s depending on c . Inserting it into Eq.(3.19) gives

$$\mathcal{L}_{\text{cINN}} = -\left\langle \log p_{\text{latent}}(\bar{G}_{\theta}(x; c)) + \log \left| \det \frac{\partial \bar{G}_{\theta}(x; c)}{\partial x} \right| \right\rangle_{p_{\text{data}}(x, c)} . \quad (3.21)$$

- *Conditional diffusion*: the condition is one more input of the denoiser, just like the time step, $\epsilon_{\theta}(x_t, t, c)$. Noise is only ever added to x , never to c , and the loss is

$$\mathcal{L}_{\text{DDPM}} = \left\langle |\epsilon - \epsilon_{\theta}(x_t, t, c)|^2 \right\rangle_{(x_0, c) \sim p_{\text{data}}, t, \epsilon} . \quad (3.22)$$

The argument of Sec. 3.2 now gives the average noise for a given noisy point *and* condition, $\epsilon_{\theta}(x_t, t, c) = \langle \epsilon | x_t, c \rangle$. Generating uses the same steps as before, with the condition kept fixed.

For generative unfolding we train on simulated pairs with the reco-level event as condition, $c = x_{\text{reco}}$, and the parton-level event as output, $x = x_{\text{part}}$. Applied to measured events, each of them can be unfolded any number of times by drawing new latent points, and the unfolded distribution is

$$p_{\text{unfold}}(x_{\text{part}}) = \int dx_{\text{reco}} p_{\text{model}}(x_{\text{part}}|x_{\text{reco}}) p_{\text{data}}(x_{\text{reco}}) , \quad (3.23)$$

which we evaluate by generating one or more parton-level events for each measured event. For inference the roles are the same, with the measured data as condition and the model parameters as output; the generated parameters are samples from their posterior. Both come with a caveat: the network learns the conditional probability of the simulation, and by Bayes' theorem

$$p(x_{\text{part}}|x_{\text{reco}}) = \frac{p(x_{\text{reco}}|x_{\text{part}}) p_{\text{sim}}(x_{\text{part}})}{p_{\text{sim}}(x_{\text{reco}})} \quad (3.24)$$

it depends on the parton-level distribution of the simulation, which acts as a prior.

[adapted from Ref. [1], Secs. 5.2 and 5.2.1, *Conditional generative networks*]

The conditional flow matching of the next subsection uses the word conditional in a different sense: there it refers to the paths, which are constructed conditional on one data point x_0 each. A physics condition can be added to it in exactly the same way as above, as one more input of the network, $v_{\theta}(x_t, t, c)$.

3.5 The same idea with straight paths: flow matching

Adding noise step by step is only one way to connect data and noise. Conditional flow matching (CFM) [4] takes the simplest possible path, a straight line from a data point x_0 to a noise point $r \sim \mathcal{N}(0, 1)$,

$$x_t = (1 - t) x_0 + t r \quad \text{with } t \in [0, 1] . \quad (3.25)$$

Along this line a point moves with the constant velocity

$$\frac{dx_t}{dt} = \frac{d}{dt} [(1 - t) x_0 + t r] = r - x_0 . \quad (3.26)$$

The network $v_\theta(x_t, t)$ is trained to predict this velocity, again with an MSE,

$$\mathcal{L}_{\text{CFM}} = \left\langle |v_\theta(x_t, t) - (r - x_0)|^2 \right\rangle_{x_0 \sim p_{\text{data}}, t, r} . \quad (3.27)$$

As for the DDPM, many lines pass through the same x_t , and the same argument as for the noise shows that the network learns the average velocity of all of them,

$$v_\theta(x_t, t) = \langle r - x_0 | x_t \rangle , \quad (3.28)$$

an arrow at every point and time that says in which direction the data lies. To generate, we draw $r \sim \mathcal{N}(0, 1)$ and follow the arrows backwards in small steps from $t = 1$ to $t = 0$. A Taylor expansion to first order in the step size gives the update

$$x_{t-\Delta t} = x_t - \Delta t \frac{dx_t}{dt} + \mathcal{O}(\Delta t^2) \approx x_t - \Delta t v_\theta(x_t, t) , \quad (3.29)$$

the Euler method for solving the ordinary differential equation $dx_t/dt = v_\theta(x_t, t)$. There is no fresh noise, so each noise point is mapped to exactly one data point, just like in a flow. The averaged paths are much straighter than the DDPM steps, so CFM needs far fewer steps, typically tens, and it has become the standard for generative networks in particle physics. [adapted from Ref. [1], Sec. 4.4.2, *Conditional flow matching*, for the conventions]

Comparing the three: the flow gives us the density and fast sampling, but needs an invertible architecture. The DDPM and CFM use any network and a simple regression loss and are more expressive, but need many network evaluations per event, and the density is no longer a by-product of the training.

Going deeper.

- The DDPM loss is not ad hoc: it follows from maximizing the likelihood $p_{\text{model}}(x_0)$ like for the flow, via a lower bound (the ELBO), which reduces to Eq.(3.7) up to weights for the individual time steps.
- The average noise is the gradient of the log-density of the noisy data, $\epsilon_\theta(x_t, t) \approx -\sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log p_t(x_t)$, called the score. Following it moves points uphill in probability. In the limit of continuous time the forward process becomes a stochastic differential equation, and DDPM sampling is its time-reversed solution.
- CFM is a continuity equation for $p(x, t)$: the averaged velocity is exactly the field that transports the data density into the Gaussian, and it also gives access to $p_{\text{model}}(x)$.

Hands-on: two-moons again, now as a density: a normalizing flow with coupling layers and a diffusion model, trained on the same data. As a bonus, a flow conditioned on the class label, $p_{\text{model}}(x|y)$, turns into a likelihood-ratio classifier like on Day 1. The notebook is available at <https://github.com/SaschaDief/grad-days-ml-tutorials>.

4 Generative networks in physics

Trained on $pp \rightarrow Z_{\mu\mu} + \text{jets}$, the CFM learns the sharp Z -peak perfectly, while the DDPM models it slightly too wide.

[adapted from Ref. [1], Sec. 4.5.1, *Factorized density*]

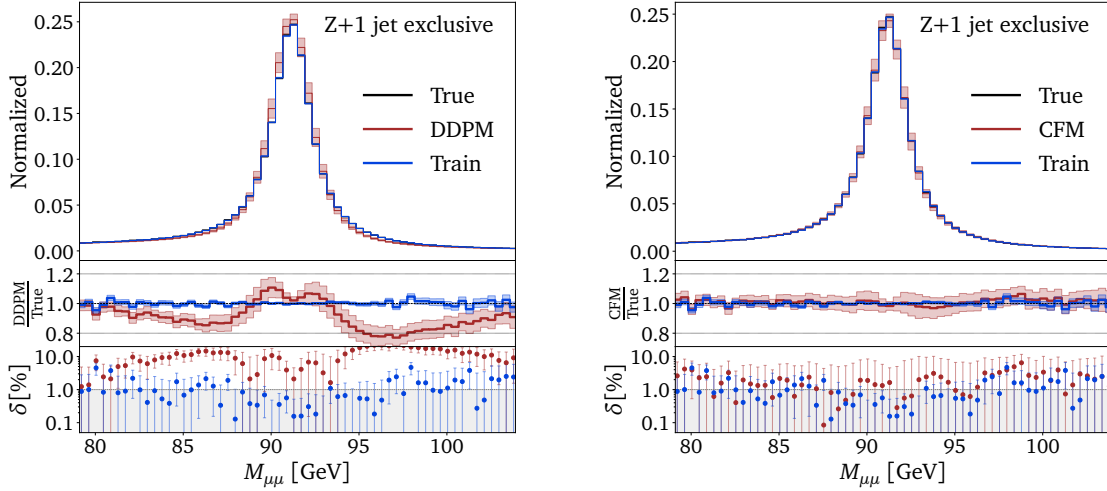


Figure 2: $M_{\mu\mu}$ distribution for $Z_{\mu\mu} + 1$ jet from the DDPM (left) and CFM (right) diffusion networks. Figure from Ref. [11].

References

- [1] T. Plehn, A. Butter, B. Dillon, T. Heimel, C. Krause, and R. Winterhalder, *Modern Machine Learning for LHC Physicists*, [arXiv:2211.01421 \[hep-ph\]](#).
- [2] L. Dinh, J. Sohl-Dickstein, and S. Bengio, *Density estimation using Real NVP*, in *International Conference on Learning Representations (ICLR)*. 2017. [arXiv:1605.08803 \[cs.LG\]](#).
- [3] J. Ho, A. Jain, and P. Abbeel, *Denoising Diffusion Probabilistic Models*, [arXiv:2006.11239 \[cs.LG\]](#).
- [4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, *Flow Matching for Generative Modeling*, [arXiv:2210.02747 \[cs.LG\]](#).
- [5] A. Butter, T. Heimel, S. Hummerich, T. Krebs, T. Plehn, A. Rousselot, and S. Vent, *Generative networks for precision enthusiasts*, *SciPost Phys.* **14** (2023) 4, 078, [arXiv:2110.13632 \[hep-ph\]](#).
- [6] ATLAS Collaboration, *AtlFast3: the next generation of fast simulation in ATLAS*, *Comput. Softw. Big Sci.* **6** (2022) 7, [arXiv:2109.02551 \[hep-ex\]](#).
- [7] C. Krause *et al.*, *CaloChallenge 2022: a community challenge for fast calorimeter simulation*, *Rept. Prog. Phys.* **88** (2025) 11, 116201, [arXiv:2410.21611 \[physics.ins-det\]](#).
- [8] A. Hallin, J. Isaacson, G. Kasieczka, C. Krause, B. Nachman, T. Quadfasel, M. Schlaffer, D. Shih, and M. Sommerhalder, *Classifying anomalies through outer density estimation*, *Phys. Rev. D* **106** (2022) 5, 055006, [arXiv:2109.00546 \[hep-ph\]](#).
- [9] M. S. Albergo, G. Kanwar, and P. E. Shanahan, *Flow-based generative models for Markov chain Monte Carlo in lattice field theory*, *Phys. Rev. D* **100** (2019) 034515, [arXiv:1904.12072 \[hep-lat\]](#).
- [10] M. Dax, S. R. Green, J. Gair, J. H. Macke, A. Buonanno, and B. Schölkopf, *Real-Time Gravitational Wave Science with Neural Posterior Estimation*, *Phys. Rev. Lett.* **127** (2021) 241103, [arXiv:2106.12594 \[gr-qc\]](#).
- [11] A. Butter, N. Huetsch, S. Palacios Schweitzer, T. Plehn, P. Sorrenson, and J. Spinner, *Jet diffusion versus JetGPT – Modern networks for the LHC*, *SciPost Phys. Core* **8** (2025) 026, [arXiv:2305.10475 \[hep-ph\]](#).